

Lecture 2 Process

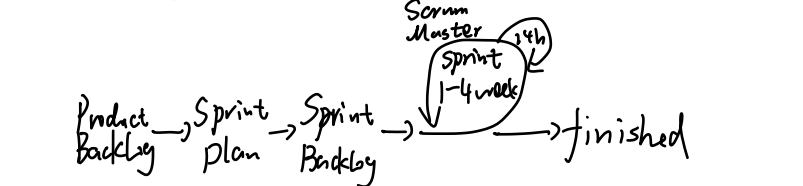
2.1 Overview: a set of phases take place in certain order
 Planning, Req, Design, Implementation, Testing, Deployment, Maintenance
 Order & Transition → template of strategy
 Continuum: Agile (plan is incremental) → plan-driven (计划驱动)

2.2 Plan driven: waterfall: req → sys design → imp → test → dev → maintain
 sequential, linear, no overlap, flows downwards, outputs are docs
 X add tight, people wait, req unclear, req change.

Incremental: each linear seq produce deliverable "increments"
 ⇒ early feedback and easier debug | X need good plan and design before
 break it down, sys struc tends to degrade as new increments +, feedback iteration

2.3 Evolutionary: iterative, evolve | 快速迭代
 (1) Prototyping: when req unclear, e.g. UI
 (2) Spiral: each spiral loop → 1 phase of the project
 Determine obj, alter, constraints | Risk analysis and alter evaluation
 Next iteration planning | Dev of deliverables
 Pro: ↓ risk, 迭代使用. Cons: heavily rely on experts, complex, costly

2.4 Agile: create and respond to change: ① work in small, consumable, increments ② req, plans, results, are evaluated continuously ③ respond to change quickly
 Scrum: Product Owner, Scrum Master, Scrum Team
 Define, adjust, accept, reject Coach the team days, tests, designers...



Sprint: short time period, deliver a potential shippable product increment.
 Product backlog: prioritize, todo list of product, story estimation pri.
 Sprint backlog: 从 PB 中选取 sprint 交付的 item. → Sprint review
 Scrum board: Story | To do | In progress | Done
 Burn down chart: tasks vs time

2.5 Dev OP: 版本分析: conflict obj "change" ↔ "stability"
 build → code plan → deploy → operate → monitor → release → test
 Principle: Collaboration, Automation, Monitor & Feedback

Lecture 3 Requirement

3.1 overview: ① what sys should do ② services provide ③ constraint on operation
 Stake holders: anyone who affects or is affected by the software
 defect cost ↑ time

3.2 Types of requirement
 Business: Why 用途, 目标
 User: who + what, little/no tech.
 System: how ① functional ② nonfunctional
 Functional: service provided, actions for spe input, behaviour for
 Non: performance, reliability, security, usability, constraints

3.3 Req analysis in Scrum.
 User story: As a < >, I want < >, So that < >
 User Need: high level, general problems from survey, interview...
 Epic ... : large feature or initiative, multiple sprints to complete
 User Story ... : 1 sprint, specific user need
 tasks ↓ : for developers, smallest unit used to track work.
 criteria acceptance: checklist of conditions mark as "done" for US

Traditional SPEC
 format: 50-page pdf
 timing: all details upfront
 owner: business analyst
 change: Formal request & revision
 SCRUM
 github issue.
 defined JIT during sprint
 PO and whole team
 Reprioritizing backlog, update acceptance criteria.

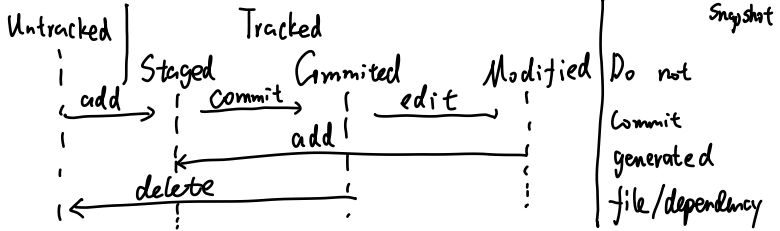
Lecture 4 Version Control System

4.1 Definition: tools help team track and manage modification to
 revisit old version, revert file, compare changes, diagnosing artifacts over time
 Local VCS: Checkout Files ← Version DB V3 V2 V1

single point of failure, no collaboration
 Centralize VCS: can cooperate, clients checkout files
 PC A ← Control VCS server
 PC B ← Version DB

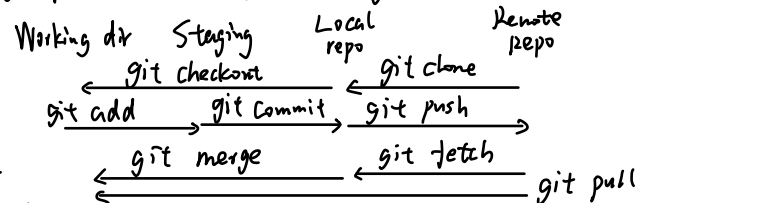
Distributed vcs: fully mirror the repo, including full history
 4.2 Git Architecture

(1) Local repo: user has full git Remote repo: host/publish code to
 features on local git repos (even X) the world let people RW.
 git init: plain project → working dir + local repo (.git)
 git add: working dir → staging area 暂存区
 git commit: staging area → local repo (.git) 暂存区快速存入本地 repo
 git checkout: local repo (.git) → working dir: pull any previous version locally
 file state: ① Untracked: not tracked ② Staged: marked a modified file to go into next commit
 ③ Committed: data in local DB ④ Modified: changed file, not committed Snapshot



(2) remote repo: origin/upstream, xxx.git
 (已有) link local → remote: git remote add <name> <URL>
 (无) git clone <URL>: 自动创建 origin, 并下载完整历史 (自动 checkout)

checks out the latest version into the working dir
 git push <remote> <local> default branch: origin ← main
 git fetch <remote>: fetch new work that has been pushed to server since last fetch
 download data to local repo only, need to git merge manually,
 git push <remote>: fetch + merge

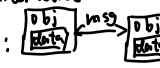


4.3 Git Internals: key-value (hash - files/dirs/commits...)
 persist (in disk) + impossible to change the contents of any file/dir
 Obj: content of file, dir, commits, tags, identified by SHA-1, in .git/objects/ without Git knowing it
 Ref: a branch, remote branch/tag, a ptr to obj, plain text, in .git/refs/
 Index: a staging area, binary files in .git/index.
 Obj: Blob: file content, by hash | Tree: list of ptr to Blob/Tree, root
 Commit: ref to top-level tree Commit: tree, parent, author, Committer
 Tag: name associated with a commit, obj, type, tagger (在 refs/tags)
 Delta based File A Δ1 → Δ2...
 Snapshot based File A A1 A2...

4.4 Git branching: Commits → parent; main → latest commit; HEAD → current.
 git merge <branch name>: branch ← main
 Fast forward: 2 branch no divergence.
 3-WAY merge: using the 2 snapshots ptr to by branch tips and the most recent common ancestor of the two
 手动编辑冲突文件, git add xxx, git commit.

4.5 Open source Contribution Workflow.
 Upstream/main fork → Origin/main clone → local repo main
 pull request origin/fix ← push hot fix

Lecture 5 Design

5.1 Architectural design
 (1) Monolithic: ① Data centered: DB in center, components delete data update
 ② Data flow (pipe and filter): input data transformed → output
 Pro: clarity, modularity Con: bottleneck, non-interactive
 ③ Call and return: sub1...sub2
 ④ Obj-oriented: 
 ⑤ Layered: Pre → Service → Business → DB 上层和下层只与相邻 (plugin 间不通信) 沟通
 (2) Service-based, distributed, DevOps (Chrome) other use plugin
 ① micro kernel (plugin): core sys only min functionality required,
 ② event-driven: event producer, processor, channel: async, distributed generate react orchestration
 ③ microservice: a suite of small services, each running on its own process

Communicating via lightweight mechanism (HTTP) * independently deployable
 Conway Law: 架构来源于团队沟通 Pro: inde, Con: complex, 不一定最优

5.2 API design: Synchronous: expect response, block to wait. Async: no block
 RESTFUL API: GET, PUT, POST, DELETE return json via URL
 { "id": 123, "name": "abc" } GET/products/123 Stateless & Synchronous
 X fetching multiple res at 1 request.

gRPC: client remote procedure call, transmit binary msg (support more of the REST)
 high performance, data-heavy. Mixed: client-facing (REST) inter-service (gRPC)
 (3) messaging: msg broker async 性, Loose coupled, 向 request/reply channel 中发/取 msg.

5.3 UI design: Visual Interaction: Clarity, accessibility, feedback
 prototyping: Low, Medium, High Fidelity
 手绘 layout interactive, detailed

Lecture 6 Build

6.1 Overview: compiling, linking software components, ext, configs.
 Automated build tool: 源码多且复杂, 构建文件分散, 外部lib过时, 多语言代码
 6.2 Types of building Sys Imperative
 (1) Task based: unit of work "task", each task is a script exe logics.
 specify other tasks as dependencies that must run before them
 build file (pom.xml) DAG, maven build life cycle, ordered build phase
 default (deploy) → clean → site, execute a build phase, runs all prior to it
 phase (abs, don't do anything by themselves)
 plugin: artifact to exe build tasks (≥ 1 goals) Goals: smallest step of maven builds
 Binding: goals are attached to phases; phase triggers goals from plugin
 Cons: ① Code exe during build ② hard to know change impact ③ no enough info build quick correct
 (2) Artifact based: producing artifacts (binary, docs) into build quick correct
 Maven: compile → test → package → install

targets: bin (直接执行), lib (供 bin/另引用) 工程师指定 targets (what) (download ext lib)
 parse BUILD → artifacts dep graph → transitive dependency → build in order
 parallelism: target → lib → only 编译 Incremental: input → output link all dep build 不重 硬 每次重建最小 artifacts set ≥ (<)

6.3 Build Artifacts (不向依赖) (向, 加API, 修bug) Bar ≥ 2.1 range =
 Unique identity artifacts: MAJOR, MINOR, PATCH 2.0.x X 3.x X
 Package everything?: ① bundle everything (ez for and user to install, Long T to Build, manage
 ② leave the dep out: Local → remote → Central 查, get from ext when build hand to dep update

Manage version: artifacts repo + build file as code, 构建时从不同下载.
 6.4 Manage dependencies
 Internal (你依赖他部分) External (ext lib) Task: 流程; Artifact: 需要产物
 Compile time, runtime, test transitive A → B * C
 P1: 隐式依赖, A → B 内找B, 外找A P2: Strict direct depend
 P3: Diamond: 统一版本 update P4: Circular: 重定向, must DAG

Lecture 7 Quality

7.1 Metrics
 (1) LOC: normalize (空行, 注释) more ≠ better 语言差异 不可比 不反映 quality
 (2) Cyclomatic complexity: logic complexity, using control-flow graph
 $V(G) = P + 1$ (# of branch nodes, if, for, while, case) $V(G) = \#E - \#N + 2$
 high Cycle ≠ low readability, 相同CC 不同可维护性

(3) OO Metrics
 Weighted Methods of Class: 类中所有方法复杂度之和, 可选 Method, LoC, 调用数
 Depth of Inheritance Tree: 类到根节点的 最大路径长度
 Number of Children: immediate subclass 数量, NOL ↑ 需测试 ↓ 缺陷: 选择
 Deps: call 方法, refer to classes, share variable
 Coupling: deps between modules X Cohesion: deps within modules ✓
 Coupling between obj classes: 不看方向, 只看 # 1-4 good
 Lack of Cohesion in Methods: R: 一对方法共享字段 P: 一对方法不共享 LCOM = P - R 越高越好

7.2 Linters: analyse src for style, syntax, potential err (Rule-based) readability
 enforce coding standards, detect errors early, ensure consistency, improve maintain
 especially useful for dynamically typed language (python, js) compiler X rule before err

7.3 Code Review
 Modification / entire new code / Auto gen code

Lecture 8 Testing

8.1 Concepts:
 Test case: sequence of action necessary to verify a specific functionality
 Lprerequisites, post condition, steps, data required for verify. or feature
 Test suite: logical grouping of test cases to run a job with diff scen. of software
 Test oracle: fault free source of expected outcome 输出正确结果和软件生 bug
 Formal specification, Business rule, Human expert (UI) Manual
 Test Quadrants:

Preventive Q2	Q4	Detective
Auto Q1	Tech Q3	Tools

 Unit test: focus on smallest unit of software (methods, classes) → isolate internal processing logic & data structure within components. parallel
 Interface, local DS, Boundary condition, inde path, error-handling path
 Integration test: Multi module interaction working together.

System test: product fulfill business specification, ready to deploy.
after integration, before deliver, env resemble production env

Test size: Process Machine Net
small 1 1 无 : sandbox fast, effective.
medium mul 1 localhost: integration of mul comp.
Large mul mul remote : end-to-end

Performance test: Load, endurance, stress, peak, volume.

User accepting test: Last phase, whether work for end user

A/B testing: multi version of a variable (by business user)

E2E ← Integration ← Unit

8.2 Test Technique → 哪些代表值测试规范, → 被测试

Black box: software specification, internal unknown, cover specified behaviour.

White box: software code, internal known, cover coded behaviour.

Black: Pro: simplicity, real result, Cons: limited coverage, incomplete

White: Pro: comprehensive, early defect detection, Cons: expertise, expensive

White: assume faulty statement is necessary for revealing a fault

Code coverage:

① Statement: $\frac{\# \text{exe state}}{\# \text{total state}}$ 每语句执行1次

② Branch: $\frac{\# \text{exe branch}}{\# \text{total branch}}$ 每个if-else是否都走 → if(x<0||x>0)

③ Condition: $\frac{\# \text{Condition that are both T and F}}{\# \text{Total Conditions}}$ 每个布尔条件都要判1次

④ Branch + condition

Black: focus on input domain

① Exhaustive (穷举): impossible

② Random: sparse failure, needle in a haystack.

③ Partition: 不同 equivalence class, valid/invalid, 每能识别
equivalence partition hypo: 1 pass/fail other pass/fail.
Boundary: test bound values

8.3 Maintainable unit test: ideally, test no change after
unless System Under Test change, would

① test via public APIs: in voke SUT in the same way user
no test on private methods, or inner DB state

② test behaviour, not methods: 不为每方法写test, 为每behavior写
1方法拆多个 behavior test

8.4 Test doubles: 代替真实 server/DB: obj or function that can
stand in for a real implementation in a test,

① Stubs: return predefined data to SUT, no real logic, only
to respond to specific inputs with specific output
used when we cannot involve obj's that answer real data.

② Fakes: working implementation, in-mem DB (hashmap),

③ Mocks: behavior verification, make assertion about how SUT
interacted with the dependency (whether the function is called) ✓
记录行为是否调用, 用3种方法

8.5 UI testing: via UI, Consistency/color/Font/Typography (readable)

Behavior of interactive elements (Button) / Functional validation: input output
Adaptability: different device, OS...

Automating browser actions selenium

Lecture 9 documentation

9.1 Types:

external: End-user: user guide
internal: Administrative: team, PM roadmap, need, status report
Developer: req, Archi, design code com, API, readme.

Self-documenting code: 有意义的目录, 文件, 类, 方法, 变量名 no magic number

Code comment: when code can't provide context alone. ⇒ intent for Why
最好自解释, 不注释 bad code 重写, comment why not what.
Comments, warnings, self-admitted tech debt, TODO (禁止敷衍)

Tests as doc: should ThrowException If Stack Is Empty

Git commit msg: context of change, why, issue #

Doc as code: docs live with code, use .md

Lecture 10 CI/CD

→ 尽早捕获有问题的变更

10.1 Continuous Integration: commit in small increments, each
change is automatically built and tested by CI server before merge.
prepare: VCS, Auto build, quality control, doc generate, Team consensus.

Commit phase: pull latest code, parallel dev, pull changes merge local,
rebuild check-tests, push to central repo, a service build product on env
CI server: tasks trigger by push, compile, linter, unit test... notify

pass/fail send back, Consistency, Automation, Scalability, Immediate feedback

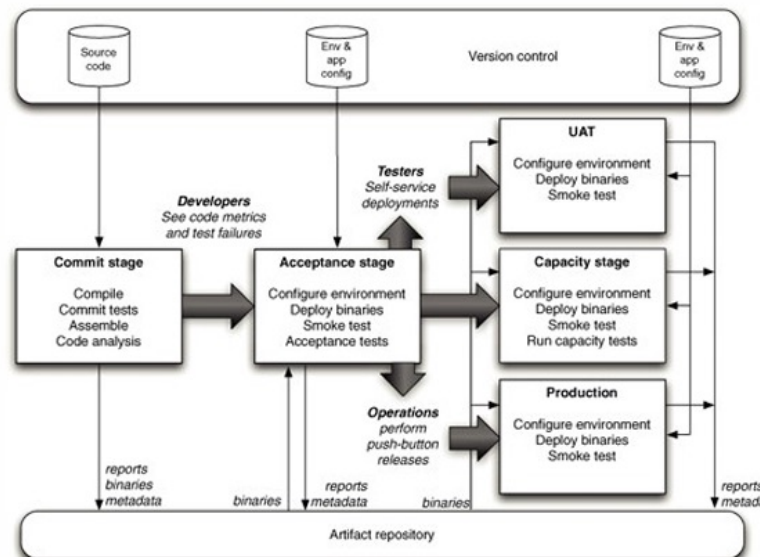
10.2 Continuous Delivery and Deployment | required: VCS, SRC, buildfile, infra,
Artifact repo: libs, plugins

Env: Dev Staging Production

Continuous delivery: build后自动到 stage 需人工 → production

Continuous deployment: build后自动到 production (无人工干预, only fail test stop)

CI/CD pipeline deployment pipeline: VCS, artifact repo, build auto tools...



Stage 1 Commit stage

Commit → VC → auto build → save bin

Stage 3 OPs team confirms and
deploy the verified → production env

Stage 2: trigger by stage 1: automated

acceptance test (integration, e2e), staging env

(Config also VC, Smoke test minimal set)

of tests run on each build to test major

typic

Minimize human err, visibility into progress of app, faster feedback.

10.3 Release: accessible for end user in the production env

deployment: available for use in staging/production env

rapid release: ① roll back ② service down ③ deploy/release strategy

Deployment strategy: expectations: ① 0 downtime ② quick to market ③ quick feedback

④ Blue green: 2 env, router to direct all traffic to diff env ⑤ 2 to roll back

Pro: min downtime, e2 fast roll back Con: redundant infra, data consistency

⑥ Rolling deploy: within server clusters, install on 1 or subset of servers at a time.

(load balancer → idle server update, remaining support traffic)

Pro: No double res, Cons: ① roll back hard ② latency all live user experience

minimal disruption ③ no control on user get which ④ uneven

③ Canary: pre selected group of user
 1. 部分 traffic → canary version → feature toggle

Pro: Real user, bug impact small, Cons: Complex traffic routing, monitoring required
 user might not be representative

10.4 Branching strategy

Trunk-based dev & release; Trunk-based dev, branch-based release
 branch-based dev, Trunk-based release (big merge, delayed integration)

Crit: Master Develop Feature Release Hotfix
 release 分支 feat 分支 pull from dev pre-release master 分支
 merge back dev (AFE → master merge 分支) master

Lecture 11 Cloud native

11.1 Deploy cmd infrastructure

Problem

- (1) die no auto migrating shell script (automate deploy)
 - (2) Monitoring ad hoc health check / restart policy
 - (3) process interfere isolation (container)
 - (4) scheduling complex Control service (K8s)
- scale up (vertical): add more res to exist node
 scale out (horizontal): add more server/nodes

Infra: Computing (hard+soft ware), Network, Data Center, Control Plane

Complexities: ① Load Balancing ② Configuration ③ Monitoring ④ Maintainance
 On-premise (本地): 自己 infra, manual intervention, long cycle, error-prone

Cloud Computing: run app on computing res by cloud provider ≠ Cloud native
 L No need manage infra, support new res, scalable, cost-effective

Cloud native: designed to build to exploit scale, resiliency, flex

Processes: DevOps, CI/CD. Architecture: Microservices

Deployment: Containers Practice: infra as code

11.2 Microservice

- (1) Language-specific Package (e.g. executable Jar → JDK → JVM)

Lack of encapsulation of tech stack. Lack of isolation
 hard to constrain the resource consumed. Manually determine the res assigned

- (2) Virtual Machine: EC2 instance → Auto scaling → load balance

Pro: good isolation (OS) Cons: Less effective res utilization, each 1 OS
 slow deployment, VM image too big

- (3) Containers: lightweight unit of software that packages an app and its dependencies together in a single environment

Physical → OS (host) → Docker Engine → Container X

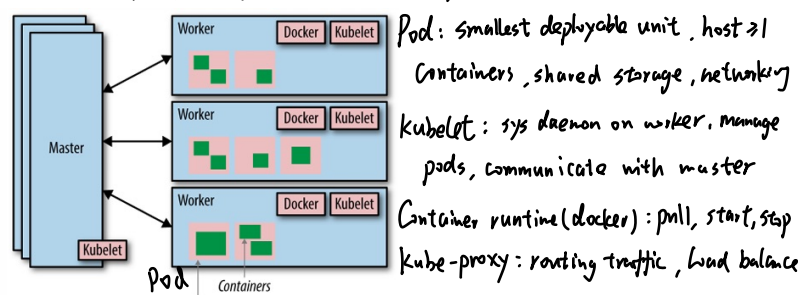
11.3 Docker: dockerfile build → docker image run → docker container

(blue print) image many layers kernel → Base → Intermediate → Container
 (read only) image push, registry pull → production env. (PUB, COPY...) (CMD, ENTRYPOINT)

Orchestration framework: 多 server 部署, 管理 res, config, health check...

Docker swarm / Kubernetes

11.4 Kubernetes: cluster = control plane + worker nodes



K8s 天生 rolling, recreate

Infrastructure as code: defining managing infra res (servers, net, std)

using code instead of manual processes. 可配置, 可自动, 可版本化

Lecture 12 Evolution

12.1 maintainance: update after delivered

- (1) Maintainability: # of bug report, Time to implement a change, impact of change

Software decay: ↑ of chaos (entropy)

- (2) Technical debt: long term cost of making short-term / suboptimal decision
 Quick and dirty fixes, well intentioned past decisions that don't scale

- (3) Refactoring: preserves the external behaviour, improve inner struct

code-smell: Blockers: 太多 code, classes, methods (Long methods/para)

DD Abuser: violate Liskov Substitution Principle (refused bequest)

L inheritance → delegation

Change preventers: single responsibility principle, 改-需改多

Coupler: methods/class highly coupled

Dispensible: 可删

Rule of 3: doing sth 3rd time → refactoring

When? add a feat, fix a bug, code review

- (4) Re-architecting: when monolithic slow, buggy, poor scalability

after 改造, 新功能, 优化其他

Strangler: 逐步, incremental, gradually refactoring

- ① implement new feat as service

- ② separate presentation tier and backend

- ③ breakup monolith by extracting functionality

12.2 System evolution

- (1) Legacy system: hard, outdate, 软件设计

Legacy code: old, 别人, 无 test, 无 doc, 难维护 ⇒ too expensive too risky

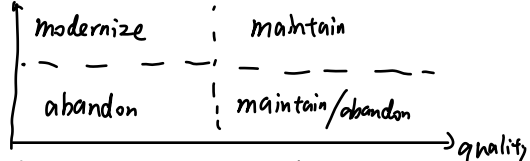
① abandon ② maintain ③ Modernize } rehost refactor re-archi

Reverse engineering: L code → design → req | replace rebuild

system transformation: old → new req → req

forward engineering: req → design → code

Business value



- (2) Deprecation

old ≠ obsolete ⇒ 有更好 alternative

warning flag

± sunset period